SISTEMAS OPERATIVOS

Aula Teórica 10

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

Scheduling in Real-Time Systems (1)

A **real-time system** is one in which **time plays na essential role**. Typically, one or more physical devices external to the computer generate stimuli, and the computer must react appropriately to them within a fixed amount of time.

Example: the computer in a compact disk player gets bits (usually a digital serial stream) as they come off the drive and must convert (DAC) them into music within a very tight time interval. If the calculation takes too long, the music will sound peculiar.

Other real-time systems are: patient monitoring in a hospital intensive-care unit; an aircraft autopilot system; robot control in an automated factory; Mars Rovers (VxWorks). In all these cases,

having the right answer but having it too late is often just as bad as not having it at all.

There are two categories:

Hard real time systems – there are absolute deadlines that must be met;

Soft real time systems – where missing na occasional deadline is undesirable, but nevertheless tolerable.

Scheduling in Real-Time Systems (2)

A **real-time system** is one in which **time plays an essential role**. Typically, one or more physical devices external to the computer generate stimuli, and the computer must react appropriately to them within a fixed amount of time.

Real-time behavior is achieved by dividing the program into a number of processes, each of whose behavior is predictable and known in advance. The processes are generally short lived and can run to completion in well under a second. When an external event are detected, it is the job of the schedular to schedule the process in such a way that all deadlines are met.

Periodic event – occurs at regular intervals; **Aperiodic event** – occur unpredictably.

A real-time system is called **schedulable** (meanning it can be implemented) if it meets the next criterion, where *m* are the number os periodic events, and event *i* occurs with period *Pi* and requires *Ci* sec of CPU time to handle each event, means that the load can be handled only if

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

Scheduling in Real-Time Systems (3)

Example:

Consider a soft real-time system with three periodic events, with periods of 100, 200, and 500 msec, respectively,

If these events require 50, 30, and 100 msec of CPU time per event, respectively the system is schedulable, because

$$\sum_{i=1}^{3} \frac{C_i}{P_i} = \frac{50}{100} + \frac{30}{200} + \frac{100}{500} = 0{,}5 + 0{,}15 + 0{,}2 < 1$$

If a fourth event with period 1 sec is added, the system will remain schedulable as long as this event does not need more than 150 msec of CPU time per event.

$$\sum_{i=1}^{4} \frac{C_i}{P_i} = \frac{50}{100} + \frac{30}{200} + \frac{100}{500} + \frac{1}{P_x} = 0{,}5 + 0{,}15 + 0{,}2 + \frac{1}{P_x} \leq 1, \qquad P_x \geq 6{,}66(6)$$

It is implicit in this calculation the assumption that context-switching overhead is so small that it can be ignored.

Scheduling in Real-Time Systems (4)

Real-time scheduling algorithms can be:

Static – an algorithm that makes their scheduling decisions before the systems start running;
Static scheduling algorithms works only when there is perfect information available in advance about the work to be done and the deadlines that have to be met.

or

Dynamic – an algorithm that makes theirs scheduling decisions at run time. After execution has started. Those algorithms do not have the static working restrictions.

Policy versus Mechanism

Up until now, we have tacitly assumed that all the processes in the system belong to different users and are thus competing for the CPU.

Sometimes it happens that one process has many children running under its control. Each children might be working on a different request or pourpose. It is also possible, that the main process has an excellent idea of which the children are the most important (or time critical) and which the least.

Unfortunely, none of the schedulers discussed earlier accept any input from users process about scheduling decisions. As a result , the scheduler rarely makes the best choice.

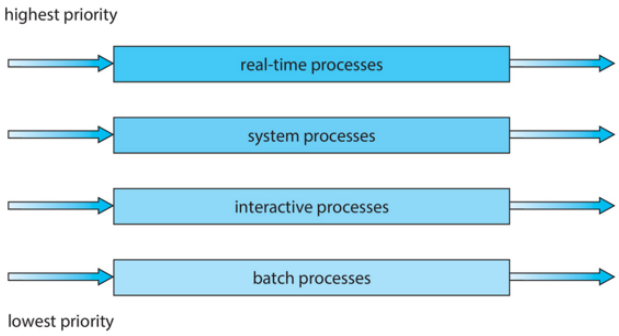
The solution for this problem is to separate the scheduling mechanism from the scheduling policy.

Example:

Suppose that the kernel uses a priority- scheduling algorithm but provides a system call by which a process can set (and change) the priorities of its children. In this way the parent can control how its children are scheduled, even though it itself does not do the scheduling. Here the mechanism is in the kernal but the policy is set by the process. Policy-Mechanism separation is a key idea.

Some systems make a clean separation between the scheduling mechanism and the scheduling policy, which allows users to have control of the scheduling algorithm.

Prioritization based upon process type

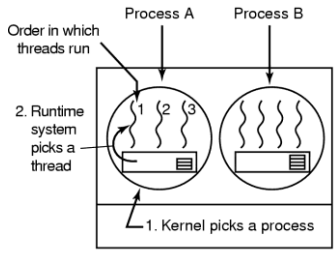


Thread Scheduling (1)

Scheduling in a system with several processes, each one with multiple threads, differs substantially depending on whether user-level threads or kernel-level threads (or both) are supported.

Possible schedule of **user-level threads**

Since the kernel is not aware of the existence of threads, it operates as it always does, picking a process, say A, and giving A control for its quantum. The thread schedukar inside A, decides which thread to run, say A1. Since thera are no clock interrupts to multiprogram threads, it may continue running as long as it wants to. If it uses up the process entire quantum, the kernel will select another to run. When process A finally runs again, thread A1 will resume running. It will continue to consume all of A's time until it is finished.



Consider the case A's threads have relatively little work to do per CPU burst, for example, 5 msec of work within a 50 msec quantum. Consequently, each one runs a little while, then yields the CPU back to the thread schedular, that might lead to the working sequence A1, A2, A3, A1, A2, A3, A1, A2, A3, A1 before the kernel switches to process B.

In pratice, round-robin scheduling and priority scheduling are the most common. The only constraint is the absence of a clock to interrupt a thread that has run too long. Since threads cooperate, this is usually not a issue.

Thread Scheduling (2)

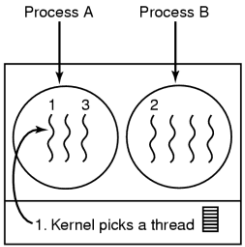
Schedule of **kernel-level threads**

Here the kernel picks a particular thread to run.

It does not have to take into account which process the thread belongs,

but it can if it wants to.

The thread is given a quantum and is forcibly suspended if it exceeds the quantum.



Possible: A1, A2, A3, A1, A2, A3
Also possible: A1, B1, A2, B2, A3, B3

With the same example, for a 50 msec process quantum and threads that block after 5 msec/CPU burst, the thread scheduling order for a 30 msec period could be:

A1, A2, A3, A1, A2, A3

or

A1, B1, A2, B2, A3, B3

or something different…

Something not possible with these parameters and user-level threads.

Thread Scheduling (3)

A major difference between user-level threads and kernel-level threads is **the performance**.

Doing a thread switch with user-level threads takes a handful of machine instructions.
With kernel threads it requires a full context switch changing the memory map and invalidating the cache, which is several orders of magnitude slower. On the other hand, with kernel-level threads, having a thread block on I/O does not suspend the entire process as it does with user-level threads.
Since the kernel knows that switching from a thread in process *A* to a thread in process *B* is more expensive than running a second thread in process *A*, it can take this information into account when making the decision.

Another important factor is that user-level threads can employ an **application-specific thread schedular**. The run-time system, knowing what all threads do, can easily pick the dispatcher to run next the best choice possible. This strategy maximizes the amount of parallelism in an environment where threads easily block with I/O.

With kerne-level threads, the kernel would never know what each thread did (although they could be assigned different priorities).

In general, an **application-specific thread scheduler** can tune an application better than the kernel can.

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

